Paradigmas de la Programación

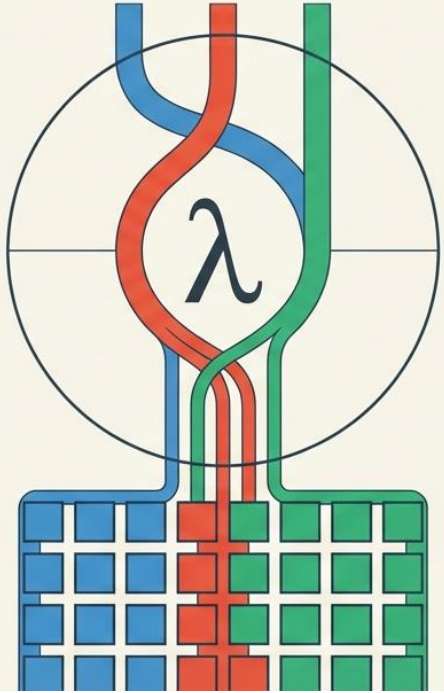


CEF.- udima

Dr. Miguel CHICCHÓN APAZA

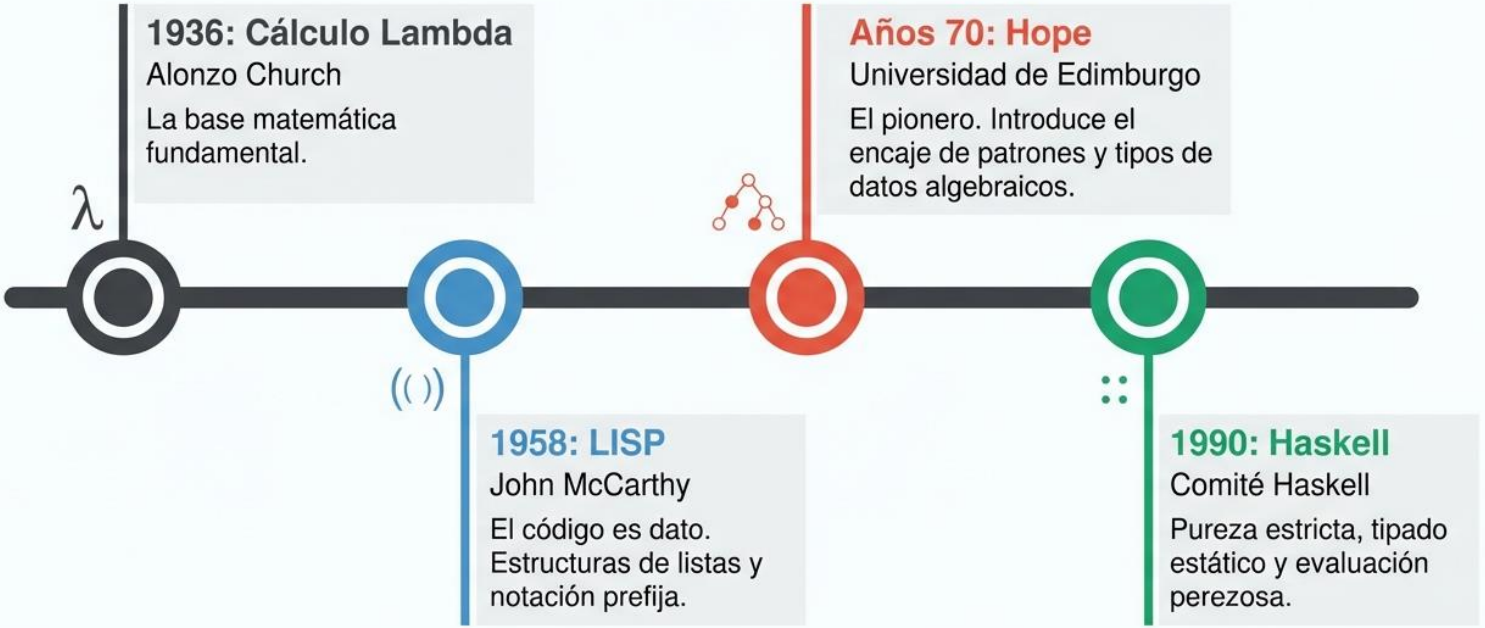
El Lenguaje Hope y el Paradigma Funcional

Una anatomía comparativa: Hope, LISP y Haskell



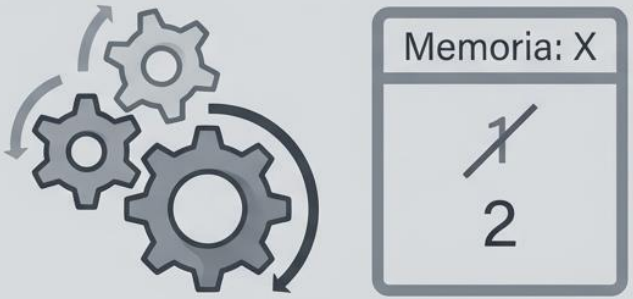
NotebookLM

La Evolución de la Lógica Pura



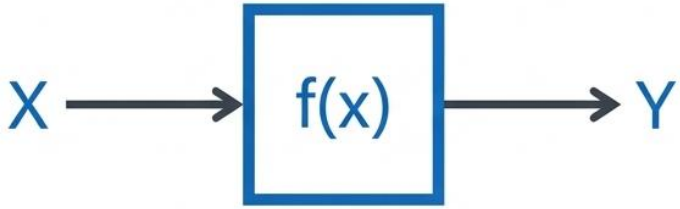
El Cambio de Paradigma: Pensar como un Matemático

Imperativo (Tradicional)



- Efectos secundarios y mutación.
- Variables que cambian de estado.
- Control mediante bucles (for/while).

Funcional (Matemático)

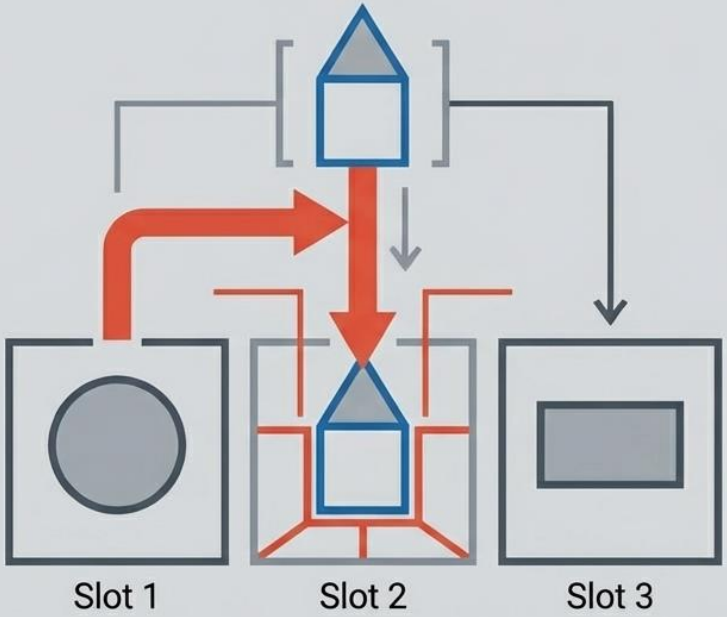


- Inmutabilidad y pureza.
- Sin estados alterados.
- Solo depende de entradas y salidas.

NotebookLM

Anatomía de Hope: El Pionero del Encaje de Patrones

Hope revolucionó la programación funcional al introducir la evaluación basada en la forma de los datos, eliminando la necesidad de condicionales complejos.



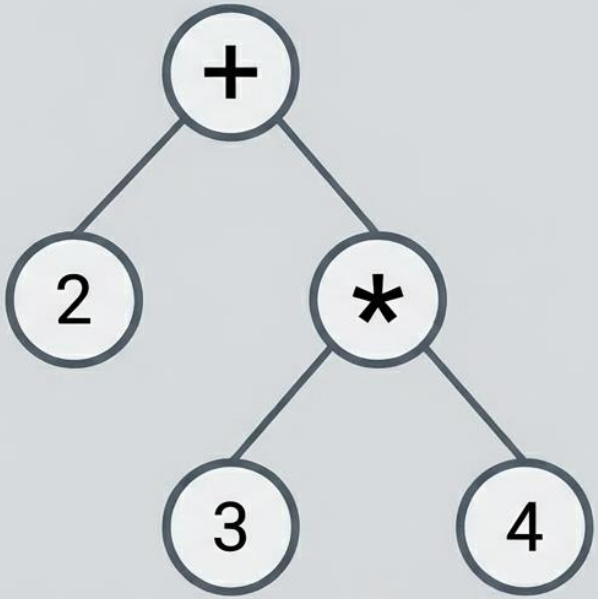

```
dec clasificar : forma → texto;

--- clasificar (circulo) <= texto1;
--- clasificar (cuadrado_con_triangulo) <= texto2;
```

NotebookLM

El Árbol de Ejecución: Sintaxis Comparada

Toda operación matemática es un árbol. Los lenguajes funcionales difieren en cómo lo escriben.



Notación Prefija

`(+ 2 (* 3 4))`

La función precede a los argumentos.

Notación Infija con Tipos

`2 + (3 * 4)` Int -> Int -> Int

Evaluación estricta clásica.

Notación Infija

`2 + (3 * 4)`

Matemáticamente puro.

NotebookLM

La Matriz de Rosetta Funcional

	LISP	Hope	Haskell

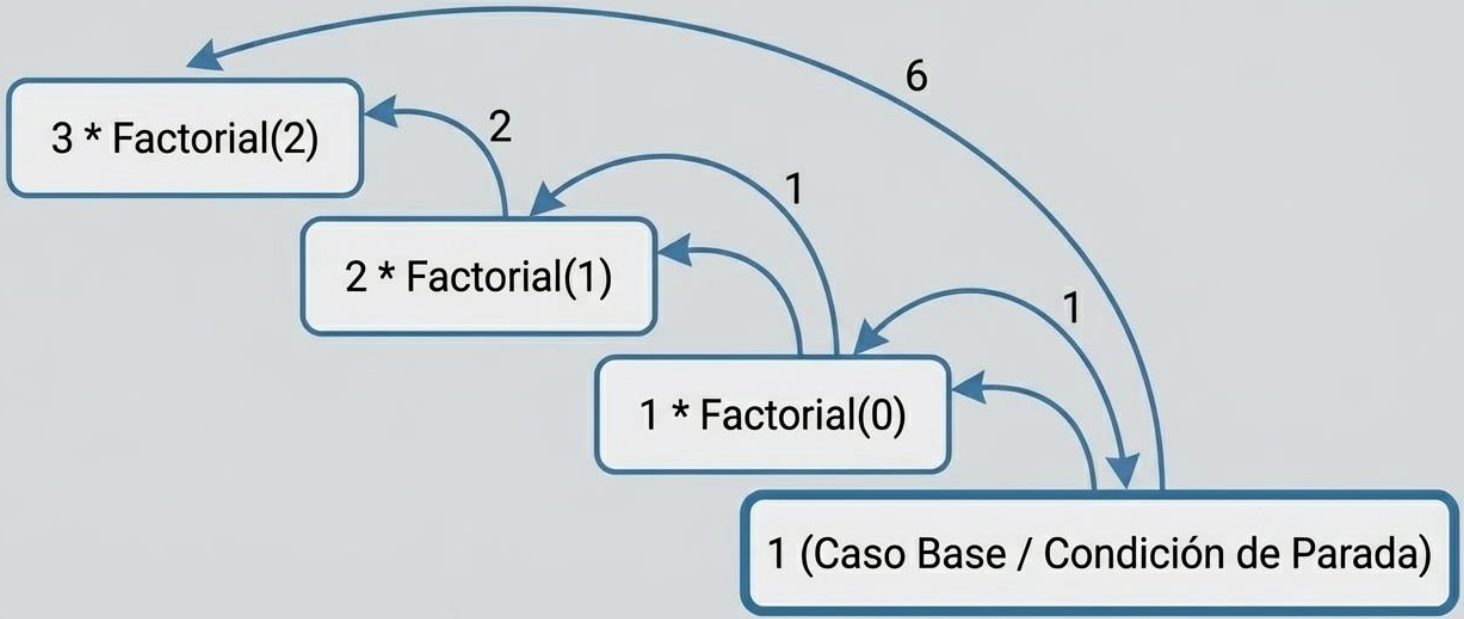
Definición de Función	`(defun suma (a b)`
	`(+ a b))`
		`dec suma : num`
		`# num -> num;`
		
		`--- suma(a,b) <= a + b;`
			`suma a b = a + b`

Sistema de Tipado	Dinámico (Evaluación en tiempo de ejecución)	Estricto y Polimórfico (Declaración explícita)	Estático e Inferido (Matemáticamente derivado)
Control de Flujo	Anidación (if, cond)	Encaje de Patrones (---)	Encaje de Patrones y Guardas (|)

NotebookLM

Capacidad 1: Recursividad como Motor de Iteración

Sin bucles tradicionales, los lenguajes funcionales iteran descomponiendo el problema hasta un caso base.



NotebookLM

Código Comparativo: Factorial

LISP

```
(defun factorial (n)
  (if (= n 0)
      1
      (* n (factorial (- n 1)))))
```

Uso de condicional explícito (if).

Hope

```
dec fact : num -> num;

--- fact 0 <= 1;
--- fact n <= n * fact(n-1);
```

Reglas de deconstrucción directas.

Haskell

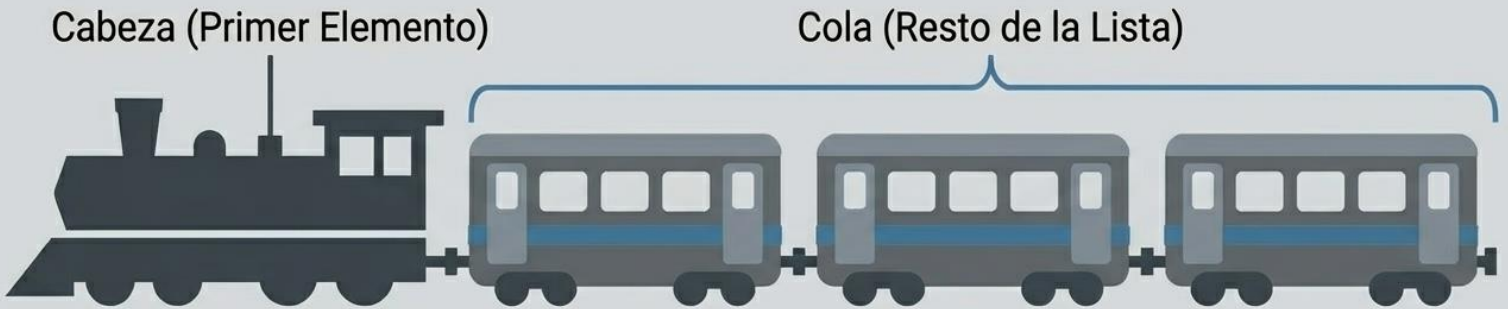
```
factorial 0 = 1
factorial n = n * factorial (n-1)
```

Enrutamiento implícito por forma.

NotebookLM

Capacidad 2: Deconstrucción de Estructuras

Los datos inmutables no se modifican en memoria. Se procesan separando el primer elemento del resto de la lista.



LISP	Hope	Haskell
Usa funciones históricas 'car' (cabeza) y 'cdr' (cola).	Usa el operador '::' (ej. x :: xs).	Usa el operador ':' (ej. x : xs).

NotebookLM

Código Comparativo: Recorriendo Listas

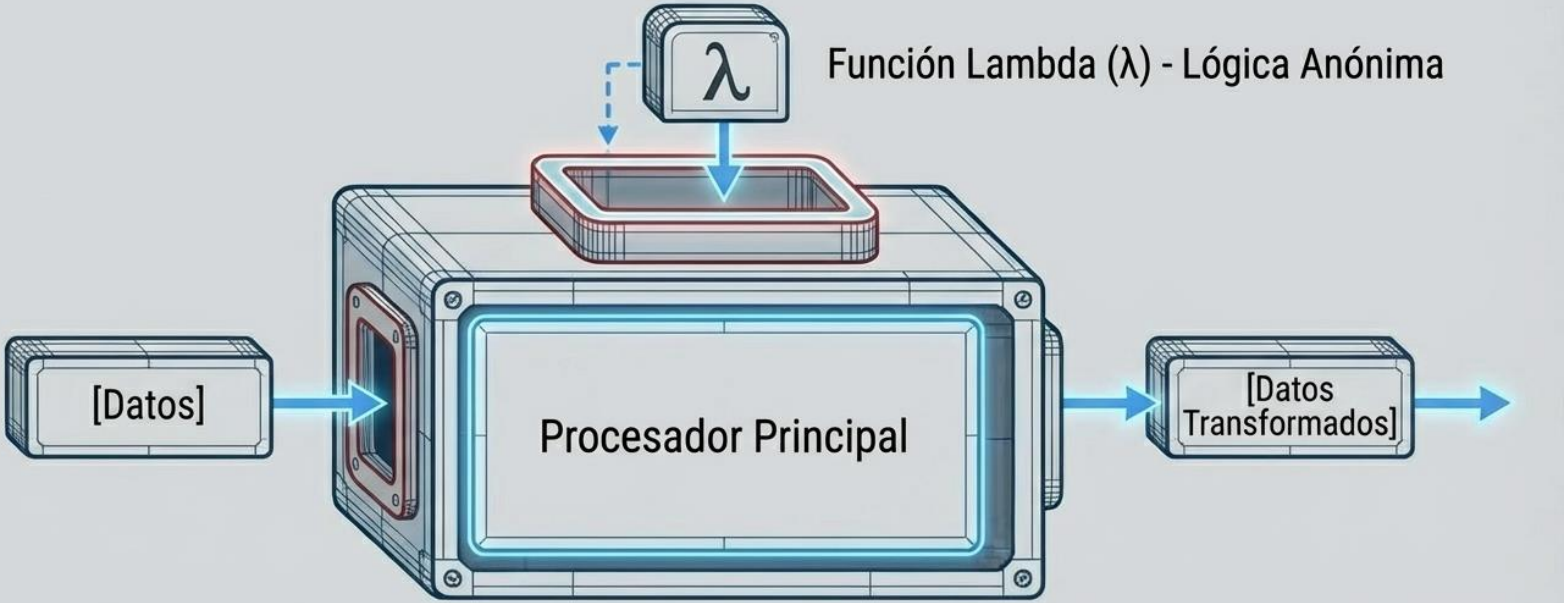
Ejemplo: Sumar 10 al primer elemento de una lista estructurada.



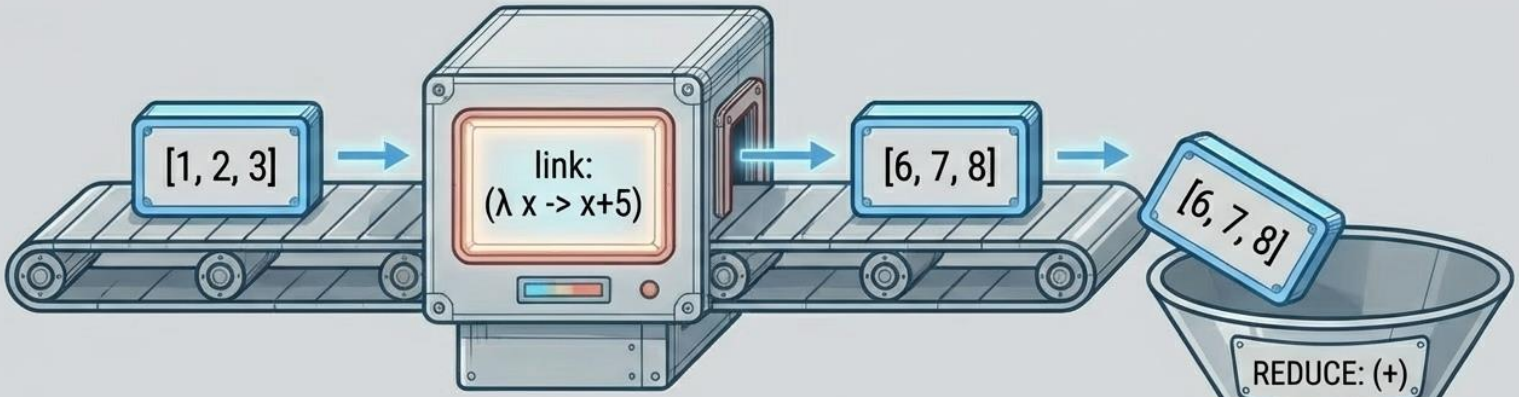
NotebookLM

Capacidad 3: Funciones de Orden Superior

Las funciones son ciudadanos de primera clase. Pueden ser pasadas como variables o insertadas como lógica intercambiable.



Transformación y Reducción: Map & Reduce

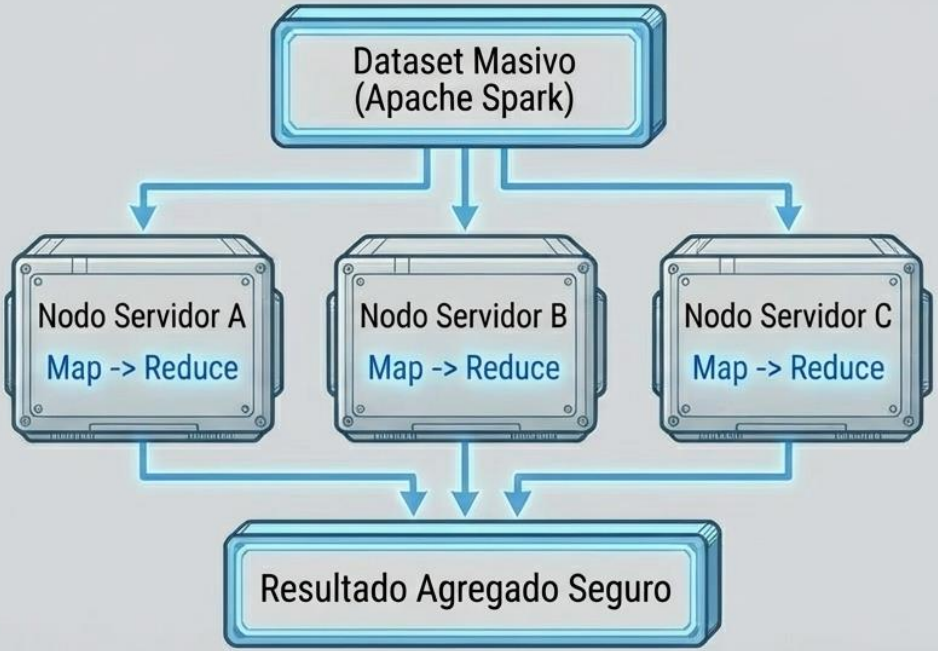


LISP	`(reduce #'+ (mapcar (lambda (x) (+ x 5)) lista))`
Hope	`reduce( +, map( lambda x => x+5, lista ) )`
Haskell	`foldl (+) 0 (map (\x -> x + 5) lista)`

NotebookLM

La Síntesis: El Legado Funcional en Big Data

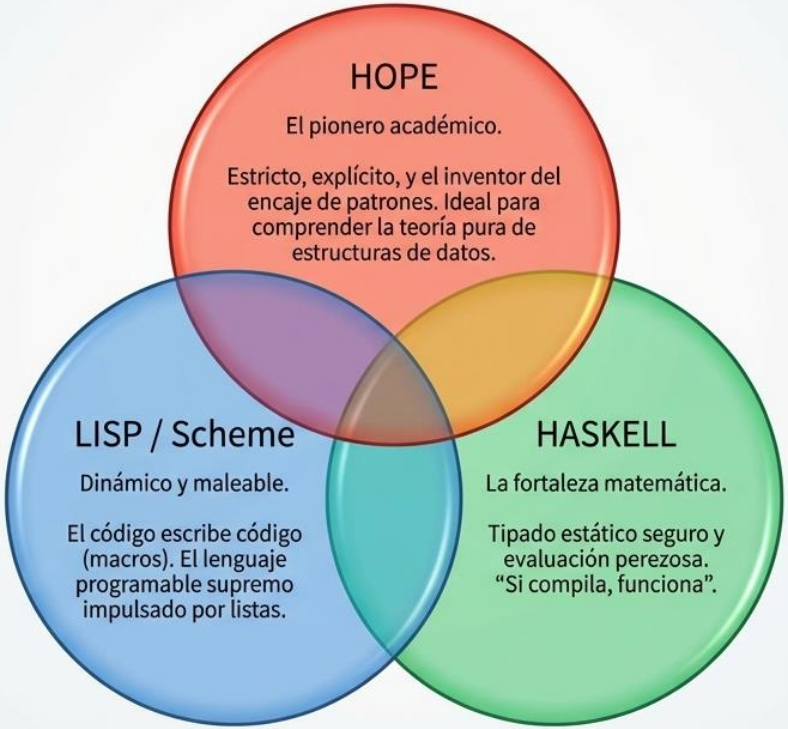
La inmutabilidad de 1958 y los patrones de los años 70 garantizan la seguridad del procesamiento distribuido moderno.



Al carecer de efectos secundarios, las evaluaciones perezosas de funciones map/reduce pueden distribuirse a través de miles de servidores. Sin estado compartido, no existen colisiones.

NotebookLM

Eligiendo el Dialecto Funcional



NotebookLM


λ
Más que un Lenguaje, una Forma de Pensar
Ya sea estructurando reglas en Hope, manipulando listas en LISP, infiriendo tipos en Haskell o procesando datos en Python, el paradigma funcional nos enseña a diseñar sistemas predecibles, inmutables y matemáticamente puros.
NotebookLM

¡Muchas gracias por vuestra atención!

www.udima.es

Juntos hacia
tu objetivo

CEF.- udima